

DC Assignment 7

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A

System Selection

Per the assignment guideline of choosing a real-life distributed system based on the first letter of my first name (Kamraan: “K”), this report analyzes **Kong Gateway**, a widely used open-source distributed API gateway and load balancer. Kong sits between client applications and backend microservices, and is used in production by companies such as Cisco, Honeywell, and Yahoo Japan to route, secure, and balance traffic across large-scale service fleets. Its core job accepting incoming requests and distributing them intelligently across a cluster of backend nodes makes it a natural case study for load balancing, task assignment, process migration, and container-based orchestration.

A Kong deployment consists of one or more **Kong nodes** (the data plane, which proxies traffic), an optional **control plane** (in hybrid mode) that pushes configuration to data-plane nodes, and a set of **upstreams** logical names representing a pool of backend **targets** (actual service instances/IP:port pairs) that requests are ultimately forwarded to. This architecture maps directly onto every theme covered in this assignment.

1. Load Balancing

Load balancing is Kong's central function: every incoming API request must be distributed across the healthy backend instances registered under an upstream, so that no single backend node is overwhelmed while others sit idle.

1.1 How Kong Distributes Workload

- **Upstream/Target model:** An administrator (or a service-discovery integration) registers multiple **targets** — backend IP:port pairs, each with a configurable weight under one logical **upstream**. Kong spreads all traffic addressed to that upstream across its live targets.
- **Multiple Kong nodes:** In production, several Kong Gateway nodes run behind an external load balancer (or a Kubernetes Service), so incoming client traffic is first spread across Kong nodes themselves, and each node then spreads its share of traffic across backend targets a two-tier balancing scheme.
- **Health checks:** Active health checks periodically probe each target, and passive health checks observe live traffic (timeouts/5xx errors) to mark unhealthy targets as “down” so load is automatically redirected away from them.
- **Hybrid-mode configuration distribution:** In hybrid deployments, a small set of control-plane nodes computes routing/balancing configuration once and pushes it to many data-plane nodes, keeping configuration consistent cluster-wide without every node hitting a shared database.

1.2 Techniques Used

- **Round-robin and weighted round-robin**, which cycle through targets in order, optionally favoring targets with higher configured weight (i.e., more capacity).
- **Least-connections**, which sends the next request to the target currently handling the fewest active connections — useful when requests have uneven processing time.
- **Consistent hashing**, which routes based on a hash of the client IP, a header, or a cookie, so requests from the same client consistently land on the same backend (session affinity).
- **Rate limiting and circuit-breaker plugins**, which prevent any single client or failing backend from monopolizing capacity, indirectly preserving balance across the cluster.

1.3 Benefits

Dimension	Benefit of Load Balancing in Kong
Performance	Spreading requests across many targets and Kong nodes increases aggregate request throughput and reduces per-node latency versus a single proxy instance.
Scalability	New Kong nodes or backend targets can be registered at runtime (via the Admin API or Kubernetes service discovery) and traffic automatically redistributes, allowing the gateway layer to scale horizontally with demand.
Fault Tolerance	Health checks continuously remove failing targets from rotation and reintroduce them once healthy again, so individual backend or node failures do not cause client-visible outages.

2. Task Assignment

Task assignment in Kong concerns how each incoming request (the “task”) is matched to a route and then handed to a specific backend target, based on current load, target health/capacity, and the request's own requirements.

2.1 Request Routing

Every request first passes through Kong's **router**, which matches the request's host, path, method, or headers against configured **Routes** to determine which **Service**/upstream should handle it. Only after this routing decision is made does the load-balancing algorithm choose a specific target within that upstream.

2.2 Balancing Algorithm Selection

Algorithm	How It Assigns Requests to Targets
Round Robin	Cycles through all healthy targets in order, giving each an equal (or weight-proportional) share of requests — simple and fair for uniform backends.

Algorithm	How It Assigns Requests to Targets
Least Connections	Tracks active connection counts per target and assigns the next request to the least-loaded one — better for backends with variable response times.
Consistent Hashing	Derives a hash key from the client IP, a request header, or a cookie so related requests are pinned to the same target, preserving session state without a shared session store.
Latency-based (with plugins/upstream health)	Deprioritizes targets showing elevated response times or recent failures, shifting more tasks to faster, healthier nodes.

2.3 Factors Considered

- **Workload:** current active-connection counts and recent response latency of each target influence least-connections and latency-aware assignment.
- **Resources:** each target's configured **weight** reflects its relative capacity (e.g., a larger VM gets a higher weight and thus a larger share of tasks).
- **Task requirements:** requests needing session affinity (e.g., a shopping-cart service) require the consistent-hashing algorithm; requests hitting rate-limited or authenticated routes are checked against plugin policies before assignment.

3. Process Migration

Process migration in Kong refers to shifting responsibility for handling traffic — either an entire Kong node's workload or a portion of an upstream's traffic — from one node/target to another while the system stays online.

3.1 Forms of Migration in Kong

- **Target migration during scaling:** When a new backend instance is added, it is registered as a new target under the upstream and immediately begins receiving a share of new requests; when an instance is scaled down, it is first marked as unhealthy/drained so in-flight requests finish before it is deregistered.
- **Blue-green and canary deployments:** Traffic is gradually shifted from an old target group (“blue”) to a new one (“green”) by adjusting target weights over time — a controlled migration of live request load between backend versions.
- **Kong node draining / rolling restarts:** Before a Kong data-plane node is restarted or removed (e.g., during an upgrade), the external load balancer or Kubernetes Service stops sending it new connections, letting existing requests complete before the node exits — migrating its share of traffic to the remaining nodes.
- **Control-plane to data-plane configuration sync:** In hybrid mode, when a new data-plane node joins the cluster, the control plane pushes the current routing/upstream configuration to it, effectively migrating the “knowledge” needed for it to take on production traffic.

3.2 Benefits

- Allows zero-downtime scaling, upgrades, and backend version rollouts, since traffic is redirected before a node/target is actually removed.
- Improves fault tolerance by moving load away from degrading nodes before they fail outright (via health-check-driven migration).
- Supports safe experimentation (canary releases) by migrating only a small, controlled fraction of traffic to new backend versions.

3.3 Challenges

- **In-flight request handling:** Care must be taken that requests already being processed by a draining node/target are allowed to complete rather than being dropped mid-flight.
- **Configuration propagation delay:** In hybrid mode, a brief delay between a control-plane change and its arrival at every data-plane node can cause temporary inconsistency in routing decisions across the cluster.
- **Session-affinity disruption:** If consistent-hashing is used for session affinity, migrating targets can change which backend a returning client is routed to, potentially invalidating local session state.
- **Operational complexity:** Deciding the pace of a blue-green traffic shift (how much weight to move and how fast) is a non-trivial tuning problem, usually automated by a deployment pipeline or service mesh.

4. Docker & Kubernetes Deployment

To demonstrate these concepts practically, Kong Gateway was containerized with Docker and deployed on Kubernetes, showing containerization, service configuration, scaling, and load distribution.

4.1 Containerization with Docker

A local Kong cluster (gateway + Postgres datastore) can be modeled with Docker Compose:

```
# docker-compose.yml
version: "3.8"
services:
  kong-database:
    image: postgres:15
    environment:
      POSTGRES_USER: kong
      POSTGRES_DB: kong
      POSTGRES_PASSWORD: kongpass
    volumes:
      - kong-db-data:/var/lib/postgresql/data

  kong-migrations:
    image: kong:3.6
    command: kong migrations bootstrap
    depends_on:
      - kong-database
    environment:
      KONG_DATABASE: postgres
```

```

    KONG_PG_HOST: kong-database
    KONG_PG_PASSWORD: kongpass

kong-gateway:
  image: kong:3.6
  container_name: kong-gateway
  depends_on:
    - kong-migrations
  environment:
    KONG_DATABASE: postgres
    KONG_PG_HOST: kong-database
    KONG_PG_PASSWORD: kongpass
    KONG_PROXY_LISTEN: 0.0.0.0:8000
    KONG_ADMIN_LISTEN: 0.0.0.0:8001
  ports:
    - "8000:8000" # proxy (client traffic)
    - "8001:8001" # admin API

volumes:
  kong-db-data:

```

Building/running: `docker-compose up -d` brings up a Postgres-backed Kong node on a single host, ready to have upstreams and targets registered through its Admin API — directly mirroring the gateway/backend-pool architecture described in Section 1.

4.2 Orchestration with Kubernetes

For production-scale deployment, Kong Gateway is run as a Kubernetes **Deployment** (multiple stateless proxy replicas), fronted by a **Service** of type LoadBalancer, with routing/upstream configuration supplied via a **ConfigMap** (DB-less declarative mode) or the Kong Ingress Controller.

```

# kong-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kong-gateway
spec:
  replicas: 3
  selector:
    matchLabels:
      app: kong-gateway
  template:
    metadata:
      labels:
        app: kong-gateway
    spec:
      containers:
        - name: kong
          image: kong:3.6
          ports:
            - containerPort: 8000
            - containerPort: 8001
          resources:
            requests:
              cpu: "250m"

```

```

        memory: "512Mi"
      limits:
        cpu: "500m"
        memory: "1Gi"
      env:
        - name: KONG_DATABASE
          value: "off"
        - name: KONG_DECLARATIVE_CONFIG
          value: "/kong/declarative/kong.yml"
      volumeMounts:
        - name: kong-config
          mountPath: /kong/declarative
      volumes:
        - name: kong-config
          configMap:
            name: kong-declarative-config
---
apiVersion: v1
kind: Service
metadata:
  name: kong-proxy
spec:
  type: LoadBalancer
  selector:
    app: kong-gateway
  ports:
    - name: proxy
      port: 80
      targetPort: 8000

```

Backend microservices behind Kong are exposed as an **upstream target group** using a Kubernetes Ingress resource managed by the Kong Ingress Controller, which automatically keeps Kong's target list in sync with the Service's live Pod endpoints — the Kubernetes-native realization of the target migration described in Section 3:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orders-service-route
  annotations:
    konghq.com/strip-path: "true"
spec:
  ingressClassName: kong
  rules:
    - http:
        paths:
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: orders-backend
                port:
                  number: 80

```

4.3 Scaling and Load Distribution

Kubernetes' native scaling capability demonstrates dynamic resource management directly analogous to Kong's own elasticity:

```
# Manually scale the Kong Gateway proxy replicas from 3 to 6
kubectl scale deployment kong-gateway --replicas=6

# Horizontal Pod Autoscaler for the Kong Gateway deployment,
# scaling proxy pods based on CPU utilization
kubectl autoscale deployment kong-gateway \
  --cpu-percent=70 --min=3 --max=10

# Backend microservice can be scaled independently;
# the Kong Ingress Controller re-syncs targets automatically
kubectl scale deployment orders-backend --replicas=8
```

When new Kong proxy pods join the Deployment, the `kong-proxy` Service immediately starts routing a share of client traffic to them (load balancing, Section 1). When the backend `orders-backend` Deployment is scaled, the Kong Ingress Controller detects the new/removed Pod endpoints and updates the corresponding upstream's target list — this is process migration (Section 3) happening natively inside the orchestrated environment. Task assignment (Section 2) then continues transparently, with each new request routed and load-balanced across whatever targets are currently registered, with no client-visible change.

5. Integrated Analysis

Load balancing, task assignment, and process migration are not independent mechanisms in Kong — they form a continuous feedback loop that keeps the deployed gateway efficient, scalable, and fault-tolerant.

5.1 How the Three Mechanisms Interact

- **Load balancing sets the target state.** Kong's upstream/target model and Kubernetes' Service layer define what an evenly distributed system should look like — traffic spread across healthy Kong nodes and, beneath them, across healthy backend targets.
- **Task assignment reaches that target for every single request.** The router and the chosen balancing algorithm (round-robin, least-connections, or consistent-hashing) decide, request by request, which node and which backend target should handle the task, based on current load, weight, and health.
- **Process migration carries out transitions when the pool changes.** Whenever a Kong node or backend target is added, removed, drained, or fails a health check, its share of responsibility is migrated to the remaining healthy members — this is the mechanism that physically realizes the rebalanced state after any change.

5.2 Efficient Resource Utilization

In the deployed system, Kubernetes resource requests/limits ensure each Kong pod gets a fair CPU/memory share of the underlying nodes, while Kong's own least-connections and weighted balancing ensure each backend target's capacity is used roughly according to its configured weight. Together, this prevents both container-level and application-level “hot spots.”

5.3 Scalability

Scalability is achieved at two layers simultaneously: Kubernetes scales the number of Kong proxy pods or backend service pods (infrastructure elasticity), while Kong's task assignment and target-migration mechanisms scale the logical distribution of requests across those pods (application-level elasticity). Neither layer alone would be sufficient — adding pods without the Ingress Controller re-syncing targets leaves the new pods idle, and rebalancing targets without adding pods leaves the cluster resource-constrained.

5.4 Fault Tolerance

If a backend pod crashes, Kubernetes restarts or reschedules it while Kong's health checks simultaneously mark the corresponding target unhealthy and redirect traffic to the remaining targets — combining infrastructure-level self-healing with application-level failover. Once the pod is healthy again, the Kong Ingress Controller re-registers it as a target, restoring full capacity and balance.

5.5 Summary

The case of Kong Gateway shows that robust distributed-system design treats load balancing, task assignment, and process migration as three phases of one continuous cycle — detect imbalance or change, decide the right routing/assignment, and migrate load to it — and that containerized orchestration platforms like Kubernetes extend this same cycle down to the infrastructure layer, making the overall system elastic, self-healing, and efficient under real-world, variable workloads.